Serverless REST API for Library System

Cloud-based library management using serverless architecture

Project Overview

This project implements a serverless, cloud-native library system that replaces traditional infrastructure-heavy designs, enabling libraries to modernize their operations without managing physical servers.

Event-Driven Architecture: It leverages event-driven computing to execute operations like book search, issue, and return. When a user performs an action (borrowing a book, returning it, or searching), Lambda functions automatically trigger to process the request, update inventory, and log transactions in real-time.

REST API Interface: REST APIs provide a standardized, language-agnostic interface between client applications and backend services. This enables seamless integration with web portals, mobile apps, and third-party systems using standard HTTP methods (GET, POST, PUT, DELETE).

Design Principles: Built using microservices principles for modularity and flexibility, allowing independent scaling and updates of different library functions without affecting the entire system.

Problem Statement

- **Traditional systems suffer from:**Tight coupling → hard to update individual modules
  Vertical scaling limits → cannot handle sudden traffic spikes
  Single point of failure → server crash leads to downtime
  Infrastructure challenges:
  Server provisioning
  Load balancing
  Maintenance costs
  Security issues:
  Weak authentication mechanisms
- **Need for:**Distributed architecture
  On-demand computing
  API-driven communication

Project Objectives

01

Design Serverless System

Build fully serverless library system using cloud services

02

Develop REST APIs

Create APIs for managing all library operations

03

Enable Real-Time Access

Provide instant access to books and user data

04

Ensure Security

Implement authentication and authorization

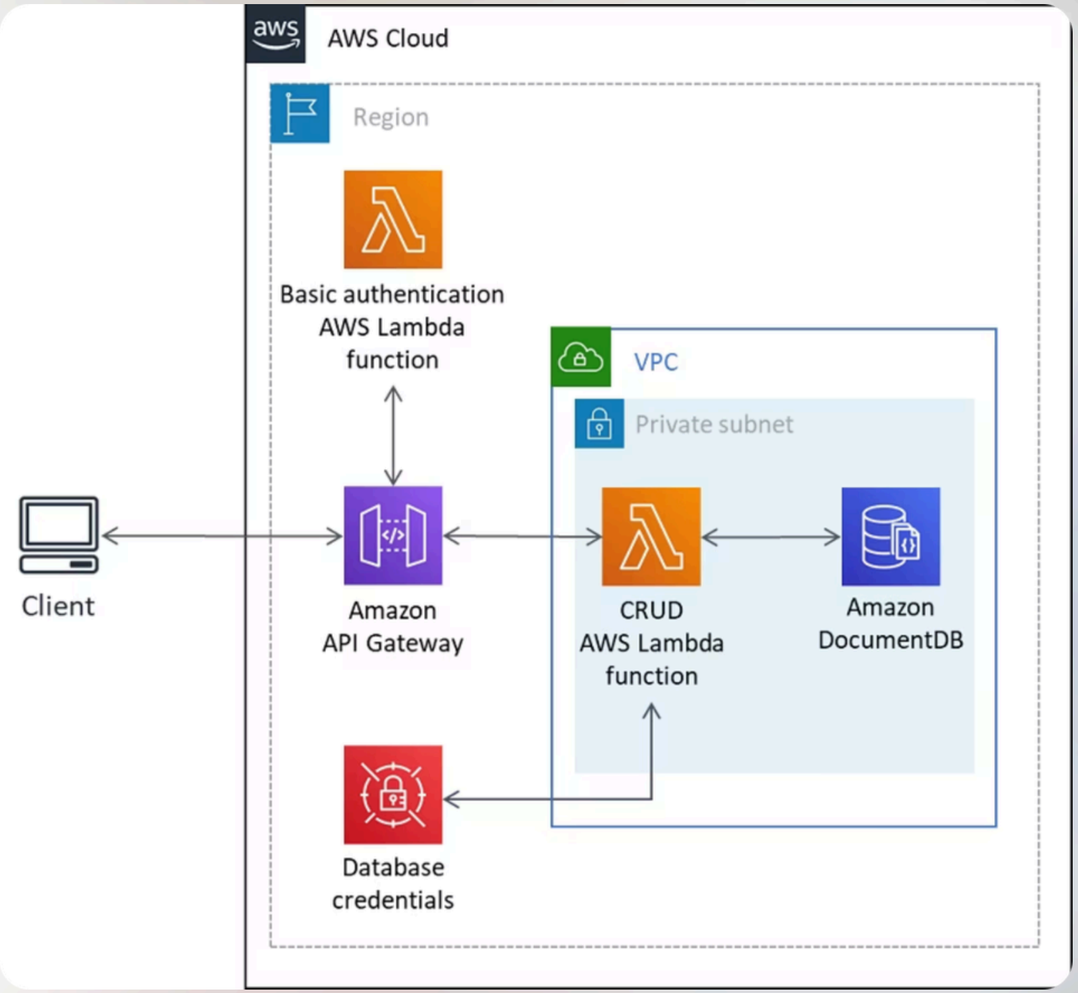
05

Optimize Performance

Reduce infrastructure cost and improve system speed

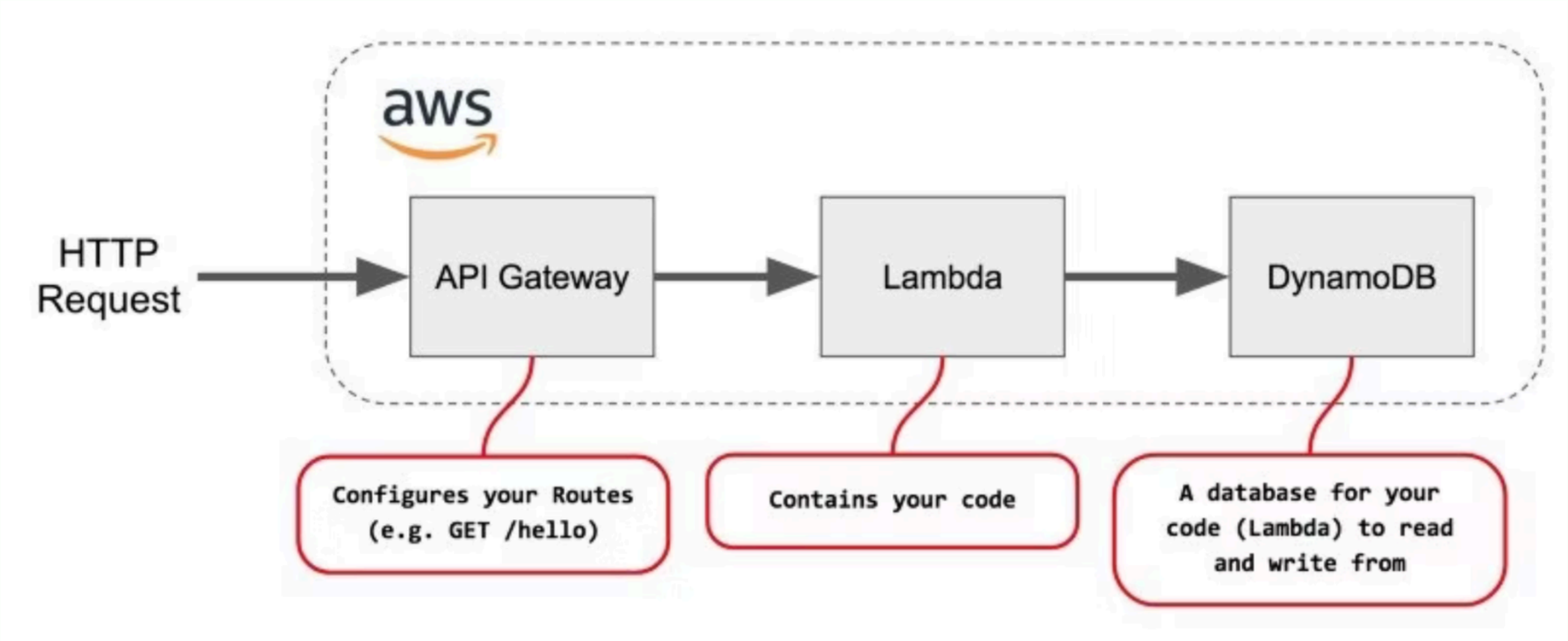
Serverless & REST API Concepts

- Serverless Computing
  Based on Function-as-a-Service (FaaS) model
  Example: Amazon Web Services Lambda
- **Key characteristics:**Stateless execution
  Event-triggered invocation
  Short-lived compute instances
  Cold Start Issue: Initial delay when function is invoked after inactivity
- **REST API Principles:**Statelessness → each request is independent
  Resource-based endpoints
  Uniform interface using HTTP methods
  Idempotency for safe operations



System Architecture

Working Flow: Client → API Gateway → Lambda → Database → Response



Cloud Services Used

API Gateway

Handles API requests and routing

Lambda

Executes backend logic without servers

DynamoDB / RDS

Stores book and user data

Cognito

Manages user authentication and authorization

CloudWatch

Monitors logs and system performance

API Endpoints & Functionality

- GET /books → Retrieve all available books
- POST /books → Add a new book to the system
- GET /books/{id} → View specific book details
- POST /borrow → Borrow a book
- POST /return → Return a borrowed book
- POST /users → Register a new user
- These APIs enable complete library management through client applications.

Scalability, Security & Advantages

- **Scalability:**Automatic scaling based on user demand
  Handles thousands of requests simultaneously
- **Security:**Authentication using Cognito / JWT
  Role-based access control (Admin/User)
  Secure communication via HTTPS
- **Advantages:**Cost-efficient (no idle servers)
  High availability and reliability
  Easy deployment and maintenance
- **Performance Optimization:**Caching using API Gateway
  Efficient database queries
  Reduced latency with serverless execution

Conclusion & Future Scope

- **Conclusion:**Serverless architecture simplifies backend development
  Eliminates infrastructure overhead
  Provides highly scalable and resilient system
- **Real-world Impact:** Digital transformation of traditional libraries
  Remote access to resources
  Reduced operational cost
- **Future Scope:**AI-based recommendation system
  Chatbot for book queries
  Integration with digital payment systems
  Analytics dashboard for usage insights